

ui.fab 全面详解（基于 NiceGUI 文档）

ui.fab 是 NiceGUI 中用于创建浮动操作按钮（Floating Action Button）的组件，基于 Quasar 的 QFab 组件实现。它以悬浮于页面之上的圆形按钮为核心，点击后可展开多个子操作按钮（ui.fab_action），适用于收纳核心功能或关联操作组，既节省界面空间，又能突出关键交互。以下从核心特性、基础用法、样式定制、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心概念

- 底层依赖：基于 Quasar 的 QFab 组件，继承其悬浮布局、展开 / 收起动画和样式体系。
- 组件构成：由「主 FAB 按钮」和「子操作按钮（ui.fab_action）」组成，主按钮控制子按钮的显示 / 隐藏。
- 核心特性：支持自定义展开方向、颜色、图标和标签，子按钮点击后默认自动关闭整个 FAB 组（可配置关闭行为）。

2. 主组件（ui.fab）初始化参数

参数名	类型	说明	默认值
icon	str	主按钮上显示的图标名称（符合 Quasar 图标规范）	-
value	bool	是否默认展开子按钮（True = 展开，False = 收起）	False
label	str / None	主按钮的可选文本标签（显示在图标旁）	None
color	str	主按钮背景色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	"primary"
direction	str	子按钮展开方向（可选值："up"、"down"、"left"、"right"）	"right"

3. 子组件（ui.fab_action）初始化参数

参数名	类型	说明	默认值
icon	str	子按钮的图标名称	-
label	str / None	子按钮的可选文本标签	None
color	str	子按钮背景色（支持 Quasar/Tailwind/CSS 颜色）	"primary"
auto_close	bool	点击子按钮后是否自动关闭 FAB 组	True

二、基础使用示例

1. 最简 FAB 组（默认配置）

主按钮带图标和标签，子按钮点击后触发通知并自动关闭 FAB 组：

```
from nicegui import ui

# 主 FAB: 图标为 navigation, 标签为 Transport, 默认向右展开
with ui.fab('navigation', label='Transport'):
    # 子操作按钮: 图标+点击事件
    ui.fab_action('train', on_click=lambda: ui.notify('Train selected'))
    ui.fab_action('sailing', on_click=lambda: ui.notify('Boat selected'))
    ui.fab_action('rocket', on_click=lambda: ui.notify('Rocket selected'))

ui.run()
```

效果：页面显示带「navigation 图标 + Transport 标签」的悬浮按钮，点击后向右展开 3 个子按钮，点击子按钮触发通知并自动收起 FAB 组。

2. 自定义颜色与展开方向

修改主 FAB 和子按钮的颜色，设置展开方向为向上，同时调整主 FAB 的位置：

```
from nicegui import ui

# 主 FAB: 购物车图标+Shop 标签, 青色背景, 向上展开, 居中偏移布局
with ui.fab('shopping_cart', label='Shop', color='teal', direction='up') \
    .classes('mt-40 mx-auto'): # Tailwind 类: 上外边距 40、水平居中
    # 子按钮: 自定义颜色和标签
    ui.fab_action('sym_o_nutrition', label='Fruits', color='green', on_click=lambda:
ui.notify('Fruits'))
    ui.fab_action('local_pizza', label='Pizza', color='yellow', on_click=lambda:
ui.notify('Pizza'))
    ui.fab_action('sym_o_icecream', label='Ice Cream', color='orange', on_click=lambda:
ui.notify('Ice Cream'))

ui.run()
```

效果：主 FAB 为青色，点击后向上展开 3 个不同颜色的子按钮，每个子按钮带图标和文本标签。

三、核心功能与进阶用法

1. 控制 FAB 展开 / 收起状态

通过 `open()`、`close()`、`toggle()` 方法手动控制 FAB 组的显示状态，或监听 `on_value_change` 事件反馈状态变化：

```
from nicegui import ui

# 创建 FAB 并赋值给变量, 用于手动控制
```

```

fab = ui.fab('menu', label='Controls')
with fab:
    ui.fab_action('settings', on_click=lambda: ui.notify('Settings'))
    ui.fab_action('help', on_click=lambda: ui.notify('Help'))

# 监听 FAB 展开/收起状态变化
fab.on_value_change(lambda e: ui.notify(f'FAB {"opened" if e.value else "closed"}'))

# 手动控制按钮
with ui.row().classes('mt-4'):
    ui.button('Open FAB', on_click=fab.open)
    ui.button('Close FAB', on_click=fab.close)
    ui.button('Toggle FAB', on_click=fab.toggle)

ui.run()

```

效果：点击「Open FAB」强制展开子按钮，「Close FAB」强制收起，「Toggle FAB」切换状态，状态变化时触发通知。

2. 禁用子按钮自动关闭

通过 `auto_close=False` 配置子按钮，点击后不关闭 FAB 组，方便连续操作多个子按钮：

```

from nicegui import ui

with ui.fab('edit', label='Edit', direction='down'):
    # 点击后不关闭 FAB，支持连续操作
    ui.fab_action('bold', label='Bold', auto_close=False, on_click=lambda:
ui.notify('Bold'))
    ui.fab_action('italic', label='Italic', auto_close=False, on_click=lambda:
ui.notify('Italic'))
    ui.fab_action('underline', label='Underline', on_click=lambda: ui.notify('Underline'))
# 默认关闭

ui.run()

```

效果：点击「Bold」「Italic」后 FAB 组保持展开，点击「Underline」后自动收起。

3. 数据绑定（动态修改属性）

通过 `bind_*` 系列方法，将 FAB 的图标、标签、颜色等属性与数据对象绑定，实现动态更新：

```

from nicegui import ui

class AppState:
    def __init__(self):
        self.fab_icon = 'menu'
        self.fab_label = 'Menu'
        self.fab_color = 'primary'

state = AppState()

```

```
# 绑定图标、标签和颜色
fab = ui.fab(
    icon=state.fab_icon,
    label=state.fab_label,
    color=state.fab_color
).bind_icon(state, 'fab_icon') \
  .bind_label(state, 'fab_label') \
  .bind_color(state, 'fab_color') # 颜色绑定继承自 BackgroundColorElement

with fab:
    ui.fab_action('refresh', on_click=lambda: setattr(state, 'fab_icon', 'refresh'))
    ui.fab_action('rename', on_click=lambda: setattr(state, 'fab_label', 'Updated Menu'))
    ui.fab_action('red', on_click=lambda: setattr(state, 'fab_color', 'red'))

ui.run()
```

效果：点击子按钮可动态修改主 FAB 的图标、标签和颜色，无需手动调用 `update()`。

4. 样式定制与布局调整

通过 `classes`、`props`、`style` 自定义 FAB 的外观和位置，支持 Tailwind/Quasar 样式体系：

```
from nicegui import ui

# 自定义样式：圆形主按钮、阴影、固定在右下角，子按钮带圆角
with ui.fab('add', color='#6AD4DD', direction='up') \
    .props('rounded-full shadow-lg') \
    .classes('fixed right-8 bottom-8'): # Quasar props: 圆形、大阴影 # Tailwind 类: 固定在右下角
    ui.fab_action('photo', label='Take Photo', color='blue-500').classes('rounded-full')
    ui.fab_action('video', label='Record Video', color='purple-500').classes('rounded-full')
    ui.fab_action('file', label='Upload', color='green-500').classes('rounded-full')

ui.run()
```

效果：主 FAB 为圆形、带阴影，固定在页面右下角，子按钮也为圆形，各自使用不同颜色。

四、核心属性与方法

1. 主组件 (ui.fab) 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（禁用后无法展开子按钮）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
icon	BindableProperty	主按钮图标（可绑定数据动态修改）
label	BindableProperty	主按钮标签（可绑定数据动态修改）

属性名	类型	说明
value	BindableProperty	展开状态 (True = 展开, False = 收起, 可绑定数据)
visible	BindableProperty	是否可见 (可绑定数据)
color	BindableProperty	背景色 (继承自 BackgroundColorElement, 可绑定数据)

2. 主组件 (ui.fab) 关键方法

(1) 状态控制

- `open()`: 展开子按钮
- `close()`: 收起子按钮
- `toggle()`: 切换展开 / 收起状态
- `disable()`: 禁用 FAB (主按钮不可点击, 子按钮无法展开)
- `enable()`: 启用 FAB
- `set_enabled(value: bool)`: 设置启用状态 (True/False)
- `set_visibility(visible: bool)`: 设置可见性

(2) 属性修改

- `set_icon(icon: str | None)`: 动态修改主按钮图标
- `set_label(label: str | None)`: 动态修改主按钮标签
- `set_value(value: bool)`: 设置展开状态 (True/False)
- `set_color(color: str)`: 动态修改主按钮背景色
- `clear()`: 移除所有子操作按钮 (ui.fab_action)
- `update()`: 触发客户端界面更新

(3) 事件与绑定

- `on_value_change(callback)`: 绑定展开 / 收起状态变化事件 (e.value 为当前状态)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `click`、`mousedown`)
- `bind_icon(target_object, target_name)`: 双向绑定图标到目标对象属性
- `bind_label_from(target_object, target_name)`: 单向绑定标签从目标对象
- `bind_visibility(target_object, target_name)`: 双向绑定可见性

(4) 其他实用方法

- `tooltip(text: str)`: 为主按钮添加悬停提示
- `delete()`: 删除 FAB 及所有子按钮
- `remove(element)`: 移除指定子按钮 (支持元素实例或 ID)
- `mark(*markers)`: 添加标记 (用于测试或元素查询)

3. 子组件 (ui.fab_action) 关键方法

- `on_click(callback)`: 绑定子按钮点击事件
- `set_icon(icon: str | None)`: 动态修改子按钮图标
- `set_label(label: str | None)`: 动态修改子按钮标签
- `set_color(color: str)`: 动态修改子按钮背景色
- `disable() / enable()`: 禁用 / 启用子按钮
- `tooltip(text: str)`: 为子按钮添加悬停提示

五、使用场景与注意事项

1. 适用场景

- 核心功能入口：如「添加」「创建」「分享」等高频核心操作，悬浮显示突出优先级。
- 关联操作组：如文本编辑（加粗、斜体、下划线）、媒体操作（拍照、录像、上传）等关联功能，收纳为子按钮。
- 移动端适配：节省移动端有限界面空间，通过展开 / 收起逻辑承载多个操作。

2. 关键注意事项

- 展开方向适配：需根据 FAB 位置选择展开方向（如右下角 FAB 适合向上 / 向左展开，避免子按钮超出屏幕）。
- 颜色优先级：Quasar 颜色与 CSS 颜色同名时（如 "red"），优先使用 Quasar 配色，自定义颜色建议使用十六进制值（如 #ff0000）。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_*` 方法的 `strict` 参数在 3.0.0+ 版本支持，使用时需确认版本。
- 层级冲突：FAB 默认悬浮于页面顶层（z-index 较高），需避免与其他悬浮组件（如弹窗、通知）层级冲突。
- 子按钮数量：建议控制子按钮数量在 3-5 个，过多会导致展开后占用过多界面空间，影响用户体验。

六、进阶示例：带权限控制的 FAB 组

结合 `bind_enabled` 和动态子按钮添加，实现基于用户权限的 FAB 功能展示：

```
from nicegui import ui

class User:
    def __init__(self, is_admin: bool):
        self.is_admin = is_admin # 是否为管理员（控制高级功能权限）

# 模拟普通用户和管理员权限
user = User(is_admin=True)

# 主 FAB
fab = ui.fab('actions', label='Actions', direction='left')
with fab:
    # 所有用户可见的基础功能
    ui.fab_action('view', label='view', on_click=lambda: ui.notify('view data'))
```

```
ui.fab_action('export', label='Export', on_click=lambda: ui.notify('Export data'))

# 管理员专属功能（根据权限动态添加）
if user.is_admin:
    ui.separator() # 分隔符
    admin_action = ui.fab_action('delete', label='Delete All', color='red',
on_click=lambda: ui.notify('Delete all data'))
    # 管理员功能可单独禁用/启用
    admin_action.bind_enabled(user, 'is_admin')

# 权限切换开关（模拟权限变更）
ui.switch('Is Admin', value=user.is_admin).bind_value(user, 'is_admin').classes('mt-4')

ui.run()
```

效果：普通用户仅显示「View」「Export」功能，管理员额外显示「Delete All」功能；切换「Is Admin」开关可动态启用 / 禁用管理员功能。